
4.2 technical-details/data-cleaning.qmd

“‘markdown

title: “Data Cleaning”

Data Cleaning and Preprocessing

Introduction to Data Cleaning

The data cleaning step is key to getting our simulated logistics info ready for exploration, modeling, or live visuals. Although we create fake events instead of pulling outside data, the stream setup still brings familiar issues, like mismatched layouts, blank entries, broken location points, or timing gaps. So cleanup turns into a repeating task, adjusting as simulations change, fresh ML traits appear, while analysis targets move. During building, this clean-up flow got reviewed again and again, making sure outputs stayed self-consistent, mathematically sound, and fit for DynamoDB rules along with later-stage ML tools.

Managing Missing Data

Because the system depends on asynchronous - like OSRM calls, sim-loop state changes, or DynamoDB saves - some data gaps showed up during tests. Missing values were identified through validation rules embedded directly in the backend. If an OSRM call dropped, for instance, the lack of path shape activated a backup straight-line router. Likewise, absent distance or speed values got caught either at truck start-up or while pulling records from DynamoDB later in the cycle.

To tackle these problems, we set up automatic backup options. When coordinates are absent, the system uses direct path estimation - while gaps in speed data fall back on a standard truck rate of 80 km/h. Estimated arrival times get recalculated whenever needed based on leftover distance and adjusted velocity figures. Since processing runs non-stop, no records are deleted; rather, reliable placeholder values keep operations stable. Instead of typical heatmap tools for spotting blanks, our model logs such events directly, noting each time defaults activate - giving engineers clear insight into performance.

Outlier Detection and Treatment

In simulations, outliers in data often come from math mistakes instead of actual real-world anomalies. For instance, OSRM might sometimes suggest unusually long route bends; similarly, ship GPS noise can generate positions beyond normal sea zones. To assess these cases, we tracked total distance patterns per planned path while ensuring truck and vessel movements stayed inside typical limits. If sudden big shifts appeared - like a marine unit jumping over half a degree in lat or lon during one reading - the model adjusted those figures to acceptable levels.

Due to outliers affecting plot precision along with model input quality, adjustments are made instead of deletions. During building stages, evaluations resembling box plots confirmed total distance trends rose steadily, while step-by-step gaps stayed inside plausible limits. Such built-in validations stop irregular points from entering outputs meant for display or analysis.

Data Type Correction and Formatting

A large part of prep work focused on keeping data types uniform throughout the system. Because DynamoDB needs numbers as Decimal objects instead of Python floats, every value for latitude, longitude, distance, or speed was carefully transformed using conditional checks prior to entry. Timestamps like `started_at` or `eta` were adjusted into ISO 8601 format through dedicated formatting rules so different parts of the application - backend, frontend, even third-party tools - could reliably use them.

Geospatial data needed precise type adjustments: since OSRM delivers coordinates as `[lon, lat]`, these were restructured into `(lat, lon)` pairs. Instead of keeping route outputs as dictionary lists, they became standard Python tuples once simulations updated. Consistent format use ensures number-sensitive tasks - like estimating arrival times, gauging leftover distances, or creating machine learning features - run without errors.

Normalization and Scaling

Even artificial data works better with normalization, particularly in machine learning contexts. Prior to integrating ML predictions into the simulation process, we checked how main inputs - like intended speed, leftover distance, or total advancement - were spread out. Leftover distance, for instance, tends to cluster low early on, then shifts toward higher values nearing destination. So that methods such as K-Means grouping and logistic prediction would run smoothly, we adjusted feature scales via z-scores while training models.

Despite small runtime adjustments in the simulation to reduce extra load, checks ensure inputs stay within normal number limits. During training, visual analysis of data before and after scaling showed extreme values didn't skew results - either for grouping or labeling patterns.

Subsetting the Data

For analysis and modeling, not all simulated fields were necessary. We subset the data to focus on features most relevant to our machine learning objectives:

- planned_speed_kmh (continuous)
- dist_remaining (continuous)
- stop_count (engineered categorical, currently zero in the simulation)
- arrival_status (binary classification label for supervised training)

By filtering down to these variables, we reduced dimensionality and ensured that the models learned from data directly tied to operational performance. Additional fields such as raw coordinates, route polyline data, and cumulative distances were retained only for visualization and geospatial logic. This separation allowed the ML models to operate efficiently without being influenced by large, high-granularity geospatial structures.

By filtering down to these variables, we reduced dimensionality while keeping inputs closely linked to real-world operations. Aside from those, details like GPS points, path lines, or total distance stayed in use - but just for maps and location-based functions. Thanks to this split, machine learning tools ran faster, avoiding interference from dense spatial datasets.

The subsetting process preserves a clean distinction between:

- High-frequency spatial data (used for the map)
- Low-dimensional operational data (used for ML)

This distinction helps keep the ML component lightweight and interpretable while maintaining full simulation fidelity.